

Operating Systems and Concurrency (G52OSC) Coursework 2025

Implement the Shortest Remaining Time First (SRTF) Process Scheduling Simulation using C Language

You are required to create a simulation program that models a Shortest Remaining Time First (SRTF) scheduling system. This algorithm is a preemptive version of Shortest Job First (SJF), where the process with the shortest remaining burst time is always selected next for execution. By simulating processes, you will create a virtual environment that mimics how an operating system schedules tasks based on remaining CPU burst times. This project will help analyze and evaluate the efficiency of the SRTF scheduling algorithm under various workloads.

Group Work

This is a Group Project with 3 members in a group.

Practical Considerations

Use any IDE (Visual Studio Code, Code::Blocks, Eclipse, etc.) or any online compiler for your implementation.

Use either single-threaded or multithreaded implementation (multithreaded approach may earn higher marks).

The program should run from a single .c file.

External sources or AI tools may be used (with proper citation in the report).

Project Breakdown and Estimated Timeline

Task	Description	Duration
Task 1	Planning, Requirements Gathering & System Design	5 days
Task 2	Core Functionality Implementation	5 days
Task 3	User Interface Development	3 days
Task 4	Implement Simulator and Visualizations (Gantt chart, diagrams)	5 days
Task 5	Testing and Debugging	1 day

Task 6

Refinement and
Documentation

3 days

Estimated Workload per Student: 60–72 hours total (10–12 hours per week)

Part A (Software Implementation) – 30%

Shortest Remaining Time First Scheduling Requirements

1. Define a structure (or class) to represent each process with attributes: Process ID, Arrival Time, CPU Burst Time, Remaining Burst Time, and Completion Time.
2. The CPU should always select the process with the shortest remaining time at any given moment. If a new process arrives with a shorter remaining time, the CPU must preempt the current process.
3. Processes must have different arrival times and simulate dynamic arrivals. Once a process finishes, it should be removed from the ready queue.
4. Implement at least three states: Ready, Running, Completed. The scheduler should update process states at each CPU time unit.
5. Your program should output: a Gantt Chart, Turnaround Time, Waiting Time, and Response Time.
6. Prompt the user to input: number of processes (max 10), arrival times, and burst times. Validate inputs for logical errors.

Sample Input Format

Enter number of processes (1–10): 4

Enter arrival and burst times:

Process 1: Arrival = 0, Burst = 7

Process 2: Arrival = 2, Burst = 4

Process 3: Arrival = 4, Burst = 1

Process 4: Arrival = 5, Burst = 4

Sample Output Format

Time	Process ID	Status	Remaining Time
0	P1	Running	6
2	P2	Ready	4
4	P3	Running	1
5	P3	Completed	0
6	P2	Running	3
...			

SRTF Performance Results:

Process 1: Turnaround = 12, Waiting = 5, Response = 0

Process 2: Turnaround = 8, Waiting = 4, Response = 2

Process 3: Turnaround = 2, Waiting = 0, Response = 0

Process 4: Turnaround = 7, Waiting = 3, Response = 1

Average Turnaround Time = 7.25

Average Waiting Time = 3.0

Recommended Approach for Students

- Step 1: Start with a simple SJF (non-preemptive) implementation.
- Step 2: Add preemption by checking for a process with a shorter remaining time at each time unit.
- Step 3: Record start, completion, waiting, and turnaround times.
- Step 4: Print results and verify with small test cases (e.g., 5 processes).
- Step 5: Add — Gantt chart, average times, and input validation.

Part B (Report) – 20%

1. Analyze your implementation results using at least 5 different process sets.
2. Compare simulation results with theoretical SRTF behavior.
3. Discuss the effect of process arrival variation on CPU efficiency and waiting times.
4. Include Gantt charts and tables summarizing averages of waiting and turnaround times.
5. Show an example of input validation for error handling (e.g., negative burst time).
6. Optionally include short code excerpts or pseudocode explaining SRTF logic.